



Gymnázium J. K. Tyla

Úvod do agentického AI a harness pro automatizovaný softwarový vývoj

ODBORNÁ PRÁCE

Autor práce: Patrik Marius, 4.D

Vedoucí práce: Michal Dočekal

Prohlášení

Prohlašuji, že jsem tuto studentskou odbornou práci vypracoval/a samostatně pod dohledem vedoucího uvedeného na první straně. Všechny použité zdroje jsou uvedeny v seznamu zdrojů a informace z nich získané jsou v textu řádně označeny odkazem na zdroj. Souhlasím s tím, aby tištěná forma práce byla uchována na Gymnázium J. K. Tyla a tam používána jako tištěný zdroj např. pro další studentské práce či pro prezentaci vzdělávání na GJKT.

V Hradci Králové dne

Podpis autora práce:

.....

Anotace

Tato odborná práce se zabývá principy agentního inženýrství (*agentic engineering*) a architekturou řídicích harnessů pro automatizovaný vývoj softwaru. Praktickým přínosem práce je návrh a implementace systému DarkFactory — agentního harnessu instalovatelného jako aplikace pro platformu GitHub (GitHub App). Systém usiluje o maximální možnou míru automatizace vývojového cyklu od sémantické analýzy požadavků v GitHub Issues, přes technické plánování, až po generování kódu a vystavení pull requestu. Práce reflektuje, že současné agentní systémy nelze vnímat jako plně autonomní: jazykové modely vyžadují deterministické mantinely proti uvíznutí v nekonečných cyklech, správu kontextu bez sémantického posunu a především kontinuální zapojení člověka formou schvalovacích bran (*Human-in-the-loop*).

Klíčová slova

agentní inženýrství, řídicí harness, GitHub App, DarkFactory, automatizace vývoje, orchestrace agentů, jazykové modely

Annotation

This thesis explores the foundational principles of agentic engineering and harness architecture for automated software development. The practical contribution of the work is the design and implementation of DarkFactory — an agentic harness installable as a GitHub App. The system aims to maximize the automation of the software engineering lifecycle, ranging from issue interpretation in GitHub Issues and technical planning to code generation and pull request delivery. The thesis emphasizes that modern agentic systems cannot be considered fully autonomous: large language models require deterministic guardrails against infinite loops, rigorous context management without semantic drift, and indispensable human-in-the-loop governance via approval gates.

Keywords

agentic engineering, harness architecture, GitHub App, DarkFactory, software development automation, agent orchestration, language models

Obsah

Prohlášení	2
Anotace	3
Klíčová slova	3
Annotation	3
Keywords	3
1 Úvod	7
1.1 Motivace a vymezení problému	7
1.2 Cíl práce a výzkumné otázky	7
1.3 Metodika práce	8
2 Teoretická část	9
2.1 Správa verzí (Git a GitHub)	9
2.1.1 Úvod do správy verzí a GitHubu	9
2.1.2 Větve (Branches) a izolace kódu	9
2.1.3 Model pull requestu (PR)	10
2.1.4 Slučování změn (Squash and Merge)	10
2.1.5 Kontinuální integrace (CI a GitHub Actions)	11
2.1.6 Požadované kontroly (Required Checks) a ochrana větví	11
2.2 LLM, chatboti, agenti a agentní inženýrství (kontext, prompt, smyčka, graf)	12
2.2.1 Úvod do velkých jazykových modelů	12
2.2.2 Tokeny, tokenizace a embedding	12
2.2.3 Tahy a správa KV cache	13
2.2.4 Kompakce kontextu a ztrátová komprese	13
2.2.5 Sémantický posun (Semantic Drift)	14
2.2.6 Alternativní paměťové architektury (RAG a stavový graf)	14
2.2.7 Degradace pozornosti (Context Rot)	14
2.2.8 Promptové inženýrství a negativní instrukce	15
2.3 Architektura řídicího harnessu a orchestrace	15
2.3.1 Úvod do řídicích harnessů	15
2.3.2 Autonomní agent vs. konverzační chatbot	15
2.3.3 Prováděcí cyklus ReAct	16
2.3.4 Běhové prostředí nástrojů a pískoviště (Sandbox)	17
2.3.5 Patologie divergence: perseverace a oscilace	17
2.3.6 Dovednosti (Skills)	18
2.3.7 Model Context Protocol (MCP servery)	18

2.3.8 Škálování: hierarchičtí subagenti a DAG workflow	18
2.3.9 Zapojení člověka do smyčky (Human-in-the-loop)	19
3 Praktická část	20
3.1 Cíl a rozsah systému DarkFactory	20
3.2 Architektura a životní cyklus požadavku	20
3.3 Orchestrační jádro a validační infrastruktura	21
3.4 Systém revizních značek pro lidský dohled nad akademickým textem	22
4 Výsledky a diskuse	23
4.1 Sjednocení pracovních postupů a přenositelnost napříč doménami	23
4.2 Provozní metriky a spolehlivost agentních běhů	23
4.3 Poznatky a systémová úskalí z provozu	23
4.4 Diskuse: Porovnání architektur	24
4.5 Systémová a metodická omezení	24
5 Závěr	25
Seznam příloh	26
A Obsah přiloženého média	27
B Schéma konfiguračního manifestu darkfactory.json	28
C Sdílené workflow pro GitHub Actions	29
D Systémové prompty plánovacího a kódovacího agenta	30
E Protokol revizních značek v sazebním systému Typst	31
Seznam zdrojů	33

Seznam obrázků a tabulek

Obrázek 1	Architektura autonomní ReAct smyčky (Reasoning + Acting) a tok dat mezi uživatelé, kontextem, modelem a výkonným prostředím.	17
Tabulka 1	Rozpoznávání prostředí podle souboru popisujícího balíček.	21

1 Úvod

1.1 Motivace a vymezení problému

V moderním softwarovém inženýrství dosáhla automatizace vysokého stupně zralosti. Sestavení zdrojových kódů, běh testovacích sad, statická analýza i nasazování do produkce probíhají běžně bez nutnosti lidského zásahu. Hlavním úzkým hrdlem celého vývojového procesu tak zůstává samotná tvorba a modifikace zdrojového kódu — časová prodleva mezi zadáním nového požadavku v podobě úkolu či hlášení chyby a vytvořením otestované, bezpečně začlenitelné změny.

Inspirací pro překonání tohoto omezení je průmyslový koncept **temné továrny** (*Dark Factory*) — plně automatizovaného výrobního provozu, který funguje samostatně bez nutnosti stálé přítomnosti lidské obsluhy. Cílem tohoto přístupu není vytlačení lidského inženýra, nýbrž posun jeho role: člověk definuje záměr, architekturu a funkční specifikaci, zatímco mechanické, rutinní a opakující se úkony přebírají autonomní systémy.

Nástup velkých jazykových modelů (LLM) otevřel cestu k automatizaci syntézy kódu, avšak stávající nástroje vykazují zásadní limity. Většina současných řešení (konverzační asistenti a doplňování kódu v editoru) řeší pouze izolovaný krok v podobě návrhu textového fragmentu. Chybí jim hlubší integrace do vývojového cyklu repozitáře: přímá práce se souborovým systémem, schopnost interpretovat výstupy překladače, iterativně odstraňovat syntaktické regrese a respektovat deterministická pravidla projektu.

Ústřední inženýrská otázka této práce proto nespočívá v tom, zda jazykový model dokáže napsat fragment kódu. Zkoumáme, jaká kontrolní a dozorčí architektura — označovaná jako **řídící harness** — musí model obklopovat, aby bylo možné jeho výstupům v produkčním repozitáři spolehlivě důvěřovat a dosáhnout vysoké míry autonomie se zachováním lidského dohledu.

1.2 Cíl práce a výzkumné otázky

Hlavní cíl: Vymezit teoretické principy agentního inženýrství (*agentic engineering*) a navrhnout modulární architekturu řídícího harnessu pro automatizovaný vývoj softwaru se zachováním lidského dohledu v klíčových rozhodovacích bodech.

Dílčí cíle:

- Vymezit infrastrukturu pro správu verzí (Git, GitHub a kontinuální integraci).
- Analyzovat limity velkých jazykových modelů (dynamiku kontextového okna, jev Context Rot, ztrátovou kompresi a sémantický posun).
- Navrhnout architekturu řídícího harnessu zahrnující nástrojové smyčky (ReAct), bezpečnostní pískoviště a hierarchickou orchestraci subagentů.
- Formalizovat mechanismy zapojení člověka do smyčky (*Human-in-the-loop*), schvalovací brány a protokol revizních značek pro dohled nad textovými výstupy.

Výzkumné otázky:

- **VO1 (Míra automatizace a role člověka):** Lze vývojový proces od zadání požadavku (GitHub Issue) po pull request strukturovat tak, aby role vývojáře spočívala výhradně v architektonickém dozoru a schvalování záměru (Human Gate), bez nutnosti ručního psaní rutinního kódu?
- **VO2 (Řízení divergence a spolehlivost smyčky):** Jakými architektonickými mechanismy lze v harnessu spolehlivě zabránit patologiím modelu (perseveraci, oscilaci a zacyklení v ReAct smyčce)?
- **VO3 (Integrita paměti a eliminace sémantického posunu):** Jak spravovat kontextové okno agenta při komplexních úlohách, aby nedocházelo k degradaci pozornosti (Context Rot) a ztrátě architektonických invariantů při kompresi?

1.3 Metodika práce

Práce má teoreticko-architektonický a inženýrský charakter. Postup sleduje strukturu inženýrského cyklu:

- **1. Koncepční analýza:** Systematické zmapování limitů autoregresivních modelů, dynamiky kontextového okna, jevu Context Rot a rozhraní nástrojů.
- **2. Architektonický návrh:** Formulace modulárního modelu řídicího harnessu, správy stavu, exekučního pískoviště, bezpečnostních pojistek a orchestrace subagentů.
- **3. Kritické zhodnocení:** Porovnání navržených principů s volnými agentními smyčkami a vymezení provozních limitů autonomního inženýrství.

2 Teoretická část

2.1 Správa verzí (Git a GitHub)

2.1.1 Úvod do správy verzí a GitHubu

Pro autonomní vývoj softwaru je spolehlivá správa verzí naprosto nezbytným základem. Jazykové modely generují kód na základě statistické pravděpodobnosti, a proto se nevyhnutelně dopouštějí chyb, logických přehmatů či regresí. Verzovací systém vytváří bezpečné a deterministické prostředí, v němž lze každou úpravu zaznamenat, otestovat a v případě selhání kdykoliv vrátit zpět k funkčnímu stavu. Namísto teoretických abstrakcí práce přímo využívá distribuovaný systém **Git** v kombinaci s platformou **GitHub**.

Klíčové komponenty infrastruktury zahrnují:

- **Distribuovaný systém Git (1):** Ukládá kompletní historii projektu v podobě jednotlivých revizí (*commitů*). Vývojář i agent pracují s plnou lokální kopií repozitáře, což umožňuje provádět změny, přepínat větve a spouštět lokální testy zcela nezávisle na síťovém připojení.
- **Platforma GitHub:** Slouží jako centrální bod pro sdílení kódu, týmovou koordinaci a automatizaci:
 - **Zadávání a sledování úkolů (Issues):** Strukturovaná textová zadání požadavků a hlášení chyb, která agentovi slouží jako výchozí specifikace úlohy.
 - **Revize změn (Pull Requests):** Uživatelské rozhraní pro přehledné zobrazení diffu, diskusi nad kódem a formální schválení člověkem.
 - **Automatizace (GitHub Actions):** Běhové prostředí pro automatické spouštění testů, linterů a překladů při každé události v repozitáři.

Agent v tomto pojetí nevystupuje jako černá skříňka s proprietárním protokolem, nýbrž jako standardní přispěvatel, který plně respektuje běžné vývojářské zvyklosti a nástroje.

2.1.2 Větvě (Branches) a izolace kódu

Základním bezpečnostním pravidlem při zapojení autonomních agentů do vývoje je striktní izolace rozpracovaného kódu. Stabilní kód v hlavní větvi (`main`) nesmí být nikdy přímo vystaven experimentům a chybám modelu. Agent proto veškeré úpravy provádí ve vyhrazených pracovních větvích odbočených ze základní linie projektu.

Tento princip přináší následující výhody:

- **Ochrana produkční větve:** Hlavní větev (`main`) reprezentuje stabilní, otestovaný stav připravený k nasazení. Přímé zapisování do této větve je zakázáno jak lidským vývojářům, tak autonomním agentům.
- **Dedikovaná větev pro každý úkol:** Agent pro každé zadání dynamicky vytvoří novou samostatnou větev (např. `task/123-oprava-parseru` či `agent/feature-auth`).

- **Izolace chyb a mezistavů:** Případné syntaktické chyby, dočasné nefunkční stavy ani neúspěšné hypotézy neovlivňují stabilitu hlavní větve ani práci ostatních vývojářů v týmu.
- **Bezpečné zahození nezdařených běhů:** Pokud se agent dostane do slepé uličky nebo vyčerpá přidělený rozpočet kroků, celou větev lze smazat jedním příkazem bez jakýchkoliv následků pro zbytek repozitáře.

Pokud se hlavní větev během práce agenta posune dopředu v důsledku jiné aktivity v repozitáři, pracovní větev agenta se musí před dokončením zaktualizovat (`git rebase` nebo `git merge`), aby byla zajištěna bezkonfliktní integrace.

2.1.3 Model pull requestu (PR)

Pull request (PR) představuje stěžejní komunikační uzel mezi autonomním agentem a lidským inženýrem. Jedná se o formální žádost o začlenění navržených změn z pracovní větve do větve hlavní. V tomto bodě se plně uplatňuje princip zapojení člověka do smyčky (**Human-in-the-loop**): agent kód samostatně navrhne a otestuje, avšak konečné rozhodnutí o jeho přijetí náleží vývojáři.

Rozhraní pull requestu integruje všechny podstatné informace na jednom místě:

- **Řádkový diff:** Vizuální srovnání původního a nového stavu, kde jsou jasně barevně odlišeny přidané, změněné a smazané řádky.
- **Strukturovaný souhrn změn:** Agent v popisu PR srozumitelně shrne, jaké úpravy provedl, jakou logiku zvolil a na které původní issue reagoval.
- **Výsledky automatických kontrol:** Přehled stavu automatizovaných testů a linterů z GitHub Actions (zelený či červený indikátor).
- **Revizní diskuse:** Možnost vývojáře přidávat komentáře k libovolnému řádku kódu, klást doplňující dotazy nebo vyžadovat přepracování konkrétních částí.

Lidský vývojář v roli revizora (Reviewer) posuzuje celkový architektonický záměr a rozhoduje o schválení, vrácení k dopracování, či zamítnutí pull requestu.

2.1.4 Slučování změn (Squash and Merge)

Způsob, jakým se změny z pracovní větve začlení do větve hlavní, má zásadní dopad na dlouhodobou udržitelnost a čitelnost repozitáře. Autonomní agent při řešení úlohy obvykle postupuje iterativní metodou pokus-omyl: upraví soubor, spustí testy, odhalí překlep a provede další drobný commit. V pracovní větvi tak vzniká dlouhá sekvence pomocných a experimentálních záznamů.

Zatímco klasický merge commit přenesení do hlavní větve veškeré dílčí commity a rebase je lineárně přeskládá, v agentním vývoji se jako optimální strategie uplatňuje **Squash and Merge**:

- **Sloučení mezikroků:** Všechny commity z pracovní větve jsou spojeny do jediného nového commitu, který je vložen do `main`.
- **Eliminace interního šumu:** Pomocné commity vzniklé při ladění testů se do hlavní větve vůbec nedostanou; historie projektu zůstává čistá a přehledná podle pravidla: jeden úkol = jeden commit.
- **Atomický návrat změn (`git revert`):** Pokud se v budoucnu ukáže, že začleněná úprava zanesla do produkce nečekanou vadu, lze celý úkol vrátit jediným atomickým příkazem bez nutnosti rozplétat desítky dílčích mezikroků.

2.1.5 Kontinuální integrace (CI a GitHub Actions)

Samotný jazykový model kód pouze generuje na základě statistických závislostí v trénovacích datech; nemá schopnost vnitřně ověřit, zda je vytvořený program syntakticky bezchybný a funkčně správný. Nezastupitelnou roli objektivního arbitra správnosti proto plní **kontinuální integrace (CI)** (2).

V rámci platformy GitHub zajišťuje kontinuální integraci vestavěný nástroj **GitHub Actions**:

- **Spouštění v čistých kontejnerech:** Každé workflow běží v nově alokovaném virtuálním prostředí se zamčenými verzemi nástrojů a závislostí, což vylučuje chyby způsobené lokálním stavem počítače vývojáře.
- **Automatická exekuce:** Integrační pipeline se automaticky spouští při každém pushi do pracovní větve i při otevření pull requestu.
- **Deterministická zpětná vazba pro agenta:** Pokud překlad nebo testy selžou, chybový protokol z terminálu je předán zpět do kontextu agenta, který na jeho základě provede informovanou opravu kódu.

2.1.6 Požadované kontroly (Required Checks) a ochrana větví

K tomu, aby byla kontinuální integrace efektivní, nestačí testy pouze spouštět — jejich úspěšné dokončení musí být systémově vynuceno. GitHub za tímto účelem poskytuje pravidla ochrany větví (*Branch Protection Rules*), která zabraňují začlenění neověřeného kódu do stabilní větve `main`.

Klíčové mechanismy ochrany zahrnují:

- **Požadované kontroly (*Required Checks*):** Seznam úloh v GitHub Actions, které musí skončit explicitním úspěchem (zelený stav), aby bylo technicky možné pull request sloučit:
 - **Statická analýza a linter:** Kontrola dodržení kódového stylu, odhalování mrtvého kódu a základních syntaktických prohřešků.
 - **Typová kontrola a build:** Jistota, že kód lze bez chyb zkompileovat a že typový systém nezaznamenal nekonzistence.

- **Automatizované testy:** Úspěšný průchod jednotkových i integračních testů ověřujících požadované chování.
- **Pravidlo deterministického výsledku:** Každá kontrola musí skončit jednoznačným výsledkem; tiché přeskočení testu nebo nejednoznačný stav sloučení zablokuje.
- **Povinné schválení člověkem:** Požadavek na explicitní autorizaci kódu lidským vývojárem dříve, než GitHub povolí sloučení do produkční větve.

2.2 LLM, chatboti, agenti a agentní inženýrství (kontext, prompt, smyčka, graf)

2.2.1 Úvod do velkých jazykových modelů

V agentním softwarovém inženýrství vystupuje velký jazykový model (LLM) jako stochastické kognitivní jádro celého systému. Z hlediska vnitřní architektury se jedná o dekodérový transformer (*Decoder-only*), jehož typickými představiteli jsou moderní modely řad Claude, GPT či DeepSeek (3). Role modelu nespočívá ve vystupování jako vševědoucí orákulum se spolehlivou znalostí okolního světa, nýbrž jako pokročilý generátor hypotéz, kódu a strukturovaných volání nástrojů řízený obdrženým kontextem.

Základní principy fungování modelu zahrnují:

- **Autoregresivní predikce:** Model zpracovává zadanou sekvenci textu a na jejím základě iterativně předpovídá nejpravděpodobnější následující symboly (tokeny).
- **Stochastická povaha:** Vzhledem k pravděpodobnostnímu vzorkování může model na totožný vstup reagovat mírně odlišně, což vyžaduje deterministické mantinely v nadřazeném řídicím harnessu.

Pro efektivní nasazení modelu do vývojového cyklu je nezbytné porozumět způsobu, jakým reprezentuje informace a jaké fyzické limity vymezují jeho operační paměť.

2.2.2 Tokeny, tokenizace a embedding

Jazykový model nepracuje přímo se znaky ani slovy v lidském slova smyslu. Vstupní text je nejprve deterministickým algoritmem převeden na číselné reprezentace, se kterými následně počítají maticové vrstvy neuronové sítě.

Tento proces zahrnuje následující pojmy:

- **Tokeny a tokenizér:** Token představuje základní diskrétní jednotku (celé slovo, slabiku či fragment znaků). Převod mezi textem a posloupností číselných tokenů zajišťuje tokenizér (nejčastěji na bázi algoritmu Byte-Pair Encoding, BPE).
- **Embedding:** Každý token je promítnut do vícerozměrného vektorového prostoru, kde geometrická vzdálenost a úhel vektorů vyjadřují sémantickou příbuznost pojmů (např. vektorová analogie král – muž + žena $\approx$ královna).

- **Jazyková asymetrie tokenizace:** Vzhledem k trénovacím datům optimalizovaným primárně pro angličtinu spotřebovávají flektivní jazyky s bohatou diakritikou (včetně češtiny) 2× až 3× více tokenů pro vyjádření téhož významu.

Z inženýrského hlediska je proto žádoucí vést systémové prompty, technické plány i komunikaci mezi nástroji v angličtině, aby se šetřila kapacita kontextu a snížila latence inference.

2.2.3 Tahy a správa KV cache

Interakce mezi modelem, uživatelem a okolním vývojovým prostředím neprobíhá spojitě, nýbrž v diskrétních krocích označovaných jako **tahy** (*turns*). Každý tah představuje jednu ucelenou výměnu zprávy, na niž systém reaguje.

Životní cyklus tahů a správa paměti zahrnují:

- **Typy tahů v agentní smyčce:**
 - **Tah uživatele či prostředí (*User Turn*):** Nové zadání úkolu nebo vnější událost.
 - **Tah modelu (*Model Turn*):** Vygenerovaná odpověď nebo strukturovaný požadavek na spuštění nástroje.
 - **Tah nástroje (*Tool Execution Turn*):** Zpětné hlášení výsledku exekuce (výpis souboru, výstup kompilátoru).
- **Správa KV cache (Key-Value Cache):** Aby inferenční engine nemusel při každém novém tahu přepočítávat celou historii od začátku, ukládá mezivýpočty klíčů a hodnot matic pozornosti do paměti.
- **Kontextové okno (*Context Window*):** Pevně limitovaná kapacita paměti modelu. Tento strop je dán hardwarovými limity GPU akceleratorů a kvadratickou složitostí plné pozornosti ($O(N^2)$ vzhledem k délce sekvence).

2.2.4 Kompakce kontextu a ztrátová komprese

Při rozsáhlejších úlohách se kontextové okno nevyhnutelně zaplní. V okamžiku, kdy objem historie dosáhne kritické hranice, musí řídicí harness přistoupit ke **kompakci kontextu** (*compaction*) — model je vyzván, aby dosavadní průběh sezení zkrátil do syntetického souhrnu, který nahradí starší část historie.

Tento proces však představuje destruktivní ztrátovou kompresi:

- **Ztráta deterministických detailů:** Model při rekurzivním zkracování vynechává přesná čísla řádků, signatury privátních funkcí, přesné cesty k souborům a doslovná chybová hlášení kompilátoru.
- **Oslabení negativních pravidel:** Explicitní zákazy (např. neměnit veřejné rozhraní API) bývají v souhrnu zevšeobecněny nebo zcela vypuštěny.
- **Konfirmační zkreslení (*Confirmation Bias*):** Model v souhrnu upřednostňuje fakta odpovídající jeho vnitřním statistickým asociacím na úkor netriviálních specifik konkrétního projektu.

2.2.5 Sémantický posun (Semantic Drift)

Opakovaná ztrátová komprese vede k závažné patologii známé jako **sémantický posun** (*Semantic Drift*). Pokud je historie sezení v dlouhém vývojovém běhu shrnována vícekrát po sobě, vzniká řetězec ztrátových transformací ($S_{k+1} = f(S_k, \Delta_k)$).

Rizika sémantického posunu spočívají v těchto jevech:

- **Efekt tiché pošty:** Drobné zkreslení či halucinace vzniklá v kole k je v kole $k + 1$ přijata jako nezpochybnitelný historický fakt.
- **Divergence modelu od reality:** Po několika cyklech komprese se vnitřní model reality agenta zcela rozejde se skutečným stavem zdrojového kódu v souborovém systému. Výsledkem je stav, kdy agent sebevědomě reportuje vyřešení úkolu, ačkoliv reálný kód zůstává v nefunkčním či neúplném stavu.

2.2.6 Alternativní paměťové architektury (RAG a stavový graf)

Aby se předešlo ztrátě informací způsobené kompakcí, moderní agentní architektury přesouvají část paměti mimo samotné kontextové okno. Namísto spoléhání se na jediný lineární textový kontext se uplatňují strukturovaná externí úložiště.

K hlavním přístupům patří:

- **Hierarchická epizodická paměť (RAG):** Ukládání doslovných protokolů nástrojů a historie úloh do externí databáze; do kontextu se selektivně injektují pouze bezprostředně relevantní fragmenty.
- **Persistentní graf stavu projektu (Project State Graph):** Udržování explicitního, strukturovaného přehledu o stavu repozitáře (seznam modifikovaných souborů, otevřené úkoly, výsledky testů a platné invarianty) mimo kontextové okno.

Díky tomu může agent kdykoliv obnovit přesný stav projektu bez závislosti na ztrátovém rekurzivním shrnování.

2.2.7 Degradace pozornosti (Context Rot)

Schopnost jazykového modelu pracovat s dlouhým kontextem nelze posuzovat pouze podle nominální velikosti okna. Ačkoliv moderní modely deklarují kapacitu statisíců tokenů, jejich schopnost efektivně vyhledávat a logicky propojovat fakta s rostoucí délkou kontextu výrazně klesá. Tento jev se označuje jako **degradace pozornosti** (*Context Rot*).

V praxi se projevuje dvěma hlavními mechanismy:

- **Lost in the Middle (4):** Pozornostní vrstvy transformeru spolehlivě vnímají informace na samém začátku a konci okna, zatímco fakta umístěná uprostřed dlouhého textu jsou často přehlížena.
- **Multi-Needle Reasoning:** Schopnost logicky provázat několik na sobě závislých informací rozptýlených napříč různými soubory; s rostoucí délkou kontextu tato schopnost prudce klesá.

Při komplexním křížovém refaktoringu ve velkém kontextu proto model často přehlédne klíčové souvislosti, které by v menším a čistším okně zpracoval bez potíží.

2.2.8 Promptové inženýrství a negativní instrukce

Základní chování agenta vymezuje **systémový prompt** (5), který definuje jeho identitu, sadu dostupných nástrojů a provozní mantinely. Při formulaci těchto pravidel však vývojáři narážejí na specifickou vlastnost autoregresivních modelů — problematické zpracování zákazů a negativních instrukcí.

Příčiny a inženýrská řešení tohoto jevu:

- **Úskalí negativních instrukcí:** Zákazy formulované negací (např. „nemazat existující testy“) modely často porušují, protože matice pozornosti (QK^T) asociativně aktivuje zakázaný pojem dříve, než autoregresní proces uplatní logický operátor negace.
- **Afirmativní formulace:** Pravidla je nutné formulovat pozitivně — namísto výčtu zákazů vymezit přesný postup a povolené mantinely chování.
- **Deterministická ochrana v harnessu:** Kde nestačí prompt, musí zasáhnout kód řídicího harnessu — například zpřístupněním testovacích souborů pouze pro čtení nebo zablokováním destruktivních operací na úrovni systémového volání.

2.3 Architektura řídicího harnessu a orchestrace

2.3.1 Úvod do řídicích harnessů

V terminologii agentního inženýrství označuje pojem **harness** (řídicí postroj) aplikační vrstvu, která obklopuje samotné inferenční jádro jazykového modelu. Samotné inferenční jádro provádí výhradně matematické maticové operace nad zadanými váhami a vektory tokenů; veškerou orchestraci, práci se soubory a řízení bezpečnosti zajišťuje harness.

Mezi klíčové funkce řídicího harnessu patří:

- **Inicializace a správa sezení:** Sestavení systémového promptu, dynamická injekce kontextu repozitáře a sledování spotřeby tokenů.
- **Běhové prostředí nástrojů:** Bezpečné spouštění příkazů v operačním systému a zpětné předávání výstupů modelu.
- **Řízení stavových přechodů a vynucování mantinelů:** Dohled nad dodržováním procesních pravidel, zastavení zacyklených běhů a vynucování lidských schvalovacích bran. Kvalita a provozní spolehlivost celého systému tak závisí v první řadě na robustnosti architektury harnessu, nikoliv pouze na samotném jazykovém modelu.

2.3.2 Autonomní agent vs. konverzační chatbot

Rozdíl mezi konverzačním chatbotem a autonomním agentem nespočívá v odlišném jazykovém modelu, ale v architektuře jeho zapojení do pracovního prostředí. Zatímco chatbot funguje pasivně jako textový rádce, agent vystupuje jako aktivní vykonavatel úkolů.

Srovnání obou přístupů:

- **Konverzační chatbot:**

- Reaguje pouze na přímé textové výzvy v uzavřeném okně chatu.
- Nemá přímý přístup k souborovému systému ani k nástrojům operačního systému.
- Uživatel musí navržený kód ručně zkopírovat, vložit do projektu a otestovat.

- **Autonomní agent:**

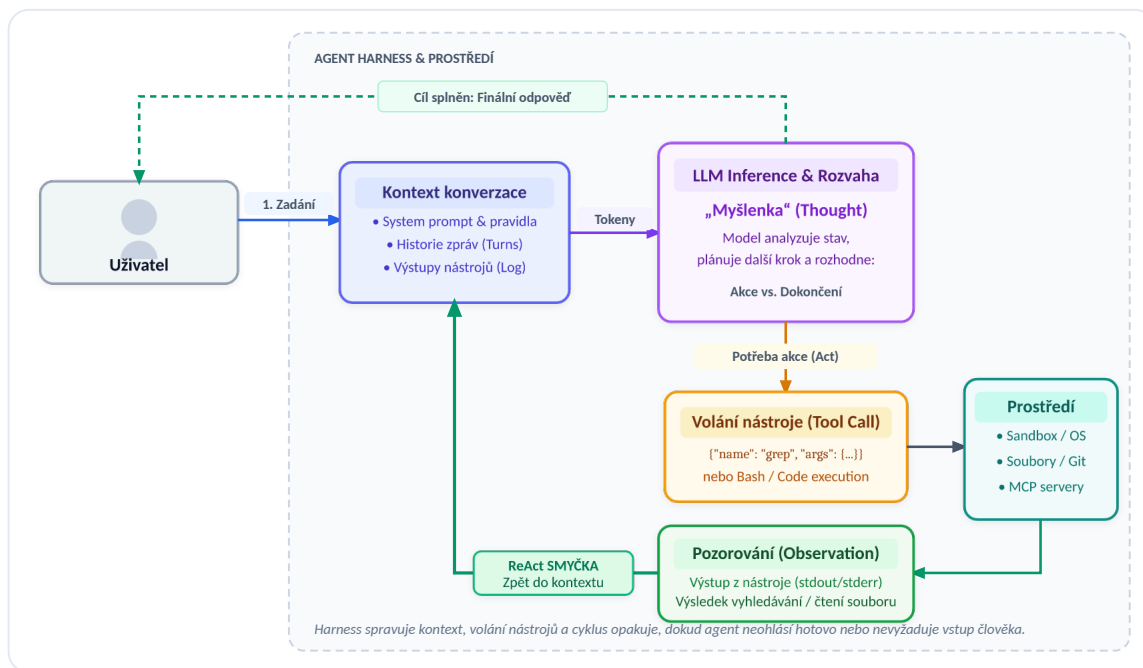
- Je vybaven sadou výkonných nástrojů (*tools*) pro práci s repozitářem.
- Aktivně prozkoumává soubory, modifikuje zdrojový kód, spouští testy a interpretuje jejich návratové kódy.
- Funguje v autonomní prováděcí smyčce, v níž iterativně reaguje na reálnou odezvu vývojového prostředí.

2.3.3 Prováděcí cyklus ReAct

Základním operačním vzorem autonomního agenta je prováděcí smyčka **ReAct** (*Reasoning + Acting*) (6). Tento vzor propojuje uvažování modelu s přímým vykonáváním akcí a interpretací jejich výsledků.

Prováděcí cyklus sestává ze čtyř navazujících fází znázorněných na Obrázek 1:

1. **Rozvaha (*Thought*):** Model vyhodnotí aktuální stav kontextu a formuluje svůj nejbližší záměr.
2. **Volání nástroje (*Tool Call*):** Emitování strukturovaného požadavku na provedení konkrétní akce s určenými parametry.
3. **Vykonání a pozorování (*Observation*):** Harness bezpečně provede akci v systému a výstup (výpis souboru či chybovou zprávu) vloží zpět do kontextu.
4. **Navazující iterace:** Model v dalším tahu analyzuje získanou odezvu a rozhoduje o navazujícím kroku.



Obrázek 1: Architektura autonomní ReAct smyčky (Reasoning + Acting) a tok dat mezi uživatelem, kontextem, modelem a výkonným prostředím.

2.3.4 Běhové prostředí nástrojů a pískoviště (Sandbox)

Aby mohl agent provádět reálné inženýrské operace, musí mu řídicí harness zpřístupnit systémové nástroje. Způsob, jakým jsou nástroje modelům předkládány, zásadně ovlivňuje ergonomii vývoje i bezpečnost celého systému.

V praxi se uplatňují dva základní modely:

- **Strukturované volání nástrojů (Tool / Function Calling):** Vstupy a výstupy jsou striktně validovány vůči formálním JSON schémátům. Zajišťuje vysokou typovou bezpečnost, avšak přináší režii tokenů spotřebovaných na definice schémat.
- **Přímé spouštění kódu (Code Execution):** Agent generuje přímo skripty (bash, Python), které harness spouští v izolovaném terminálu. Poskytuje maximální flexibilitu pro softwarový vývoj, avšak vyžaduje nekompromisní bezpečnostní izolaci.

2.3.5 Patologie divergence: perseverace a oscilace

Ponechání jazykového modelu v neomezené prováděcí smyčce vede k předvídatelným selháním. V důsledku autoregresivní povahy se v kontextu snadno vytvoří pravděpodobnostní atraktor, který model uvězní v neproduktivním cyklu.

Mezi typické patologie patří:

- **Perseverace a zacyklení:** Opakované volání identického nástroje se stejnými neplatnými argumenty (např. čtení neexistujícího souboru) i po obdržení chybové zprávy.
- **Oscilace a těkání (Thrashing):** Strídavé přepínání mezi dvěma protichůdnými zásahy (úprava modulu A rozbije modul B a následná oprava B rozbije modul A).

- **Nekontrolovaná spotřeba zdrojů (*Context Runaway*):** Rychlé vyčerpání kontextového okna i finančního rozpočtu na volání API bez dosažení cíle.

2.3.6 Dovednosti (Skills)

Se vzrůstající komplexitou úloh nelze veškeré instrukce, skripty a doménové znalosti vkládat do základního systémového promptu. K modulárnímu rozšíření schopností agenta slouží koncept **dovedností** (*Skills*).

Architektura dovedností staví na následujících principech:

- **Definiční soubor `SKILL.md`:** Dovednost tvoří adresář obsahující definiční soubor se strukturovanou hlavičkou (YAML frontmatter vymezující název a popis role) a detailním návodem k použití.
- **Dynamické načítání pro úsporu kontextu:** Do výchozího promptu se vloží pouze stručný přehled dostupných dovedností. Kompletní instrukce a skripty se do kontextu načtou až v okamžiku, kdy agent danou dovednost explicitně vyvolá.
- **Skripty a záchytné body (*Scripts & Hooks*):** Dovednosti mohou obsahovat deterministické skripty pro rutinní transformace kódu a událostní háčky vyvolávané při stavových přechodech harnessu.

2.3.7 Model Context Protocol (MCP servery)

Pro sjednocení rozhraní mezi jazykovými modely a externími nástroji či datovými zdroji vznikl otevřený standard **Model Context Protocol (MCP)** (7). Namísto vytváření proprietárních rozhraní pro každou službu definuje MCP univerzální protokol.

Základní vlastnosti protokolu MCP:

- **Protokolové rozhraní:** Komunikace probíhá prostřednictvím standardu JSON-RPC (přes standardní vstup/výstup `stdio` nebo proud událostí `Server-Sent Events / SSE`).
- **Architektonické oddělení:** Implementace nástrojů běží jako samostatný proces mimo jádro harnessu. MCP servery fungují jako znovupoužitelné komponenty, které lze snadno sdílet napříč různými agenty a projekty.

2.3.8 Škálování: hierarchičtí subagenti a DAG workflow

Monolitická agentní smyčka selhává při řešení komplexních, vícefázových úloh. Pro spolehlivé škálování se v moderních systémech uplatňuje hierarchická dělba práce a formalizace procesu do podoby grafu.

Klíčové přístupy ke škálování zahrnují:

- **Subagenti (*Subagents*):** Hlavní orchestrátor dekomponuje rozsáhlou úlohu a deleguje dílčí kroky na specializované agenty (např. průzkumník repozitáře, plánovač, kódovací dělník). Po dokončení je kontext subagenta zahozen a orchestrátor obdrží pouze čistý výsledek, což chrání primární kontext před znečištěním (*context pollution*).

- **Pracovní postupy jako grafy (DAG / Graph Engineering):** Životní cyklus požadavku je modelován jako orientovaný acyklický graf (příjem → plán → kód → testy → schválení). Hrany definují striktní závislosti (`needs`); selhání v libovolném uzlu okamžitě zastaví navazující kroky.

2.3.9 Zapojení člověka do smyčky (Human-in-the-loop)

Základním principem navrženého řešení není nekritická plná autonomie, nýbrž efektivní kooperace člověka a stroje. Autonomnímu systému náleží mechanické a rutinní úkony, zatímco klíčová architektonická a nevratná rozhodnutí zůstávají plně pod kontrolou vývojáře. Řízení lidského dohledu staví na těchto pilířích:

- **Lidské schvalovací brány (Human Gates):** Formální procesní uzly, v nichž se automatický běh pozastaví a vyčká na autorizaci operátora:
 - **1. brána (Záměr a plán):** Člověk autorizuje technický plán a rozpad požadavku dříve, než agent začne modifikovat kód v souborech.
 - **2. brána (Sémantická revize):** Člověk provádí finální kontrolu diffu v pull requestu před jeho začleněním do hlavní větve.
- **Prevence únavy z revizí (Review Fatigue):** Vyvážená frekvence kontrol — zamezení mikromanagementu na úrovni jednotlivých souborů při zachování kontroly nad celkovým architektonickým směrem.
- **Dohledatelnost původního zadání:** Trvalé uchovávání doslovného znění požadavku (GitHub Issue) bez ztrátových parafrází modelem, což brání vymizení okrajových podmínek v průběhu vývoje.
- **Transparentnost selhání a deterministická eskalace:** Zákaz tichého pohlcování chyb či halucinovaných omluv při selhání. Při vyčerpání rozpočtu nebo selhání testů harness vygeneruje strukturovaný diagnostický incident (diff, chybové hlášení, stav kontextu) a předá jej vývojáři k manuálnímu zásahu.

3 Praktická část

3.1 Cíl a rozsah systému DarkFactory

Praktickým ztělesněním teoretických principů zkoumaných v této práci je systém **DarkFactory** (8). Jedná se o agentní řídicí harness navržený jako aplikace pro platformu GitHub (GitHub App), který rozšiřuje standardní vývojářské prostředí o schopnost autonomního odbavování softwarových úkolů.

Hlavním cílem systému je maximalizace automatizace rutinních a mechanických fází vývojového cyklu:

- **Sémantická analýza zadání:** Interpretace textového popisu problému z GitHub Issues a extrakce omezujících podmínek.
- **Technické plánování:** Průzkum dotčených souborů repozitáře a návrh postupu implementace.
- **Generování a úprava kódu:** Provádění konkrétních změn v souborovém systému v izolované větvi.
- **Běh testů a samooprava:** Spouštění validačních kontrol a iterativní náprava detekovaných syntaktických regresí.
- **Vystavení pull requestu:** Otevření strukturovaného návrhu změn s vygenerovaným diffem a souhrnem.

Systém záměrně neusiluje o nekritickou plnou autonomii. Vzhledem ke stochastické povaze jazykových modelů DarkFactory staví na kooperativním modelu se zapojením člověka (*Human-in-the-loop*). Role jsou striktně rozděleny: autonomní agent obstarává kognitivně rutinní činnosti (navigaci v kódu, tvorbu dílčích funkcí a opravy chyb z testů), zatímco lidský inženýr si ponechává výhradní kontrolu nad architektonickým záměrem, schvalováním plánů a finální sémantickou revizí v pull requestu.

3.2 Architektura a životní cyklus požadavku

Architektura systému DarkFactory vychází ze striktního oddělení sdílené orchestrační logiky od specifické konfigurace jednotlivých klientských repozitářů. Veškeré výpočetní operace probíhají v hermetickém běhovém prostředí*, což zaručuje reprodukovatelnost výsledků nezávisle na okolním stavu.

Životní cyklus zpracování každého požadavku je formalizován jako stavový automat (orientovaný acyklický graf), který provádí požadavek následujícími fázemi:

- **1. Příjem zadání:** Detekce nového nebo aktualizovaného úkolu v GitHub Issues.
- **2. Technické plánování:** Průzkum repozitáře a sestavení dekompozice změn.

*Pojem **hermetické prostředí** označuje výpočetní prostředí zcela izolované od nekontrolovaných stavů hostitelského systému a sítě. Veškeré nástroje, knihovny a závislosti jsou explicitně uzamčeny na konkrétních verzích, což zaručuje determinismus a reprodukovatelnost.

- **3. Lidská schvalovací brána 1:** Pozastavení běhu a autorizace navrženého plánu vývo- jářem.
- **4. Autonomní implementace:** Editace souborů a úprava kódu v dedikované větvi.
- **5. Validací pipeline:** Běh automatických testů v CI s možností automatické samoopravy chyb.
- **6. Vystavení PR:** Otevření pull requestu s vygenerovaným popisem a řádkovým diffem.
- **7. Lidská schvalovací brána 2:** Finální kontrola a schválení člověkem.
- **8. Integrate:** Automatické sloučení do hlavní větve metodou *Squash and Merge*.

Pro zajištění univerzální použitelnosti systém implementuje dvouúrovňovou detekci pro- jektů:

- **Prostředí (*Environment*):** Určuje vyžadované binární nástroje a balíčkovací manažery podle detekovaných souborů (viz Tabulka 1).
- **Doména (*Domain*):** Určuje způsob řízení výstupů (doména programového kódu vs. textová sazba a dokumentace).

Soubor	Prostředí	Doména
<code>pyproject.toml</code>	Python	kód
<code>Cargo.toml</code>	Rust	kód
<code>package.json</code>	Node	kód
<code>go.mod</code>	Go	kód
<code>typst.toml</code>	Typst	text
<code>.latexmkrc</code>	LaTeX	text
<code>lakefile.toml</code>	Lean	matematika

Tabulka 1: Rozpoznávání prostředí podle souboru popisujícího balíček.

Všechny automaticky rozpoznané parametry lze v klientském repozitáři deklarativně pře- psat nebo rozšířit pomocí konfiguračního manifestu (např. o explicitní cesty k písmům nebo dodatečné integrační kontroly).

3.3 Orchestrační jádro a validační infrastruktura

Výkonné jádro orchestrátoru zajišťuje koordinaci mezi klientským repozitářem, jazykovými modely a validační infrastrukturou. Spolehlivost a odolnost systému v nepřetržitém provozu garantují následující operační mechanismy:

- **Rotace poskytovatelů LLM:** Úkoly jsou definovány nezávisle na formátu konkrétního poskytovatele. Při vyčerpání kvóty nebo výpadku API primárního modelu harness auto- maticky přepne na navazující model v rotačním žebříčku, což eliminuje prostoje pipeline.
- **Deterministické ověřování změn:** Každý zásah do kódu podléhá požadovaným kont- rolám v CI. Úlohy končí explicitním stavem, čímž brání zablokování větve v důsledku přeskočených kontrol.

- **Správa nastavení jako kód (*Config as Code*):** Pravidla ochrany větví, konfigurace linterů i projektové nástěnky (GitHub Projects) jsou definovány deklarativně a podléhají verzování v gitu.
- **Automaticky odvozovaná živá dokumentace (*Living Documentation*):** Dokumentace a specifikace jsou generovány přímo ze zdrojového kódu a strukturovaných inline komentářů s automatickou validací v CI.

3.4 Systém revizních značek pro lidský dohled nad akademickým textem

Protokol vizuálních revizních značek pro formát Typst formalizuje dohled člověka nad generovaným textem:

1. Postranní revizní panely (*Callout*):

- `#note[...]` (zelený): Konstruktivní náměty na vylepšení, doplnění diagramů a praktické vazby.
- `#issue[...]` (červený): Věcné nesrovnalosti, logické mezery, překlepy a duplicitní obsah.
- `#alert[...]` / `#struct-alert[...]` (žlutý): Strukturální nevyváženost, chybějící sekce a osnovové neshody.
- `#critique[...]` (oranžový): Hloubková oponentura, slepá místa, zpochybnění neověřených předpokladů k obhajobě.
- `#blue-note[...]` (modrý): Metodické vymezení, rozsah práce a konceptuální mantinely zadání.

2. Textové revizní funkce v toku textu:

- **Neověřený koncept** (`#draft[...]` / `#unconfirmed[...]`): Žluté podbarvení — koncept čekající na posouzení a revizi autorem.
- **Nově přidáný text** (`#added[...]`): Zelené podbarvení — nově vygenerovaný text začleněný agentem.
- **Potvrzený text** (`#confirmed[...]`): Modré podbarvení — text potvrzený uživatelem, čekající na finální integraci.
- **Navrženo k odstranění** (`#removed[...]`): Červené přeškrtnutí — zastaralý text navržený ke smazání.
- **Srovnávací diff** (`#diff(old, new)`): Červený původní text přeškrtnutý následovaný zeleným novým textem.
- **Čistý neoznačený text**: Finální, autorsky schválený text v hlase autora bez podbarvení.

4 Výsledky a diskuse

4.1 Sjednocení pracovních postupů a přenositelnost napříč doménami

Jedním z klíčových přínosů navržené architektury je možnost sjednotit automatizované pracovní postupy napříč různorodými technologickými ekosystémy. Místo vytváření proprietárních ad-hoc skriptů pro každý projekt zvlášť staví řešení na centralizovaném řídicím harnessu:

- **Centralizace pracovních postupů:** Klientské repozitáře neudrží vlastní izolované skripty; exekuci delegují na centrální znovupoužitelné šablony a specifičnost projektu vymezují v deklarativním manifestu.
- **Údržbová složitost $O(1)$:** Bezpečnostní oprava či aktualizace v harnessu je provedena jednou a spotřebitelské projekty ji přebírají posunem připnuté verze ($O(1)$ oproti $O(N)$ manuálním úpravám v každém repozitáři zvlášť).
- **Statická kontrola jádra:** Jádro harnessu podléhá striktní typové kontrole a jednotkovému testování před každým vydáním.
- **Omezení jediného bodu selhání (SPOF):** Projekty odkazují na neměnný kryptografický SHA hash commitu či sémantický tag, což vylučuje nechtěné regresní změny.
- **Konzistentní agentní prostředí:** Jednotný životní cyklus požadavků, správa kontextu, rotace modelů i vyhodnocení validačních bran napříč všemi projekty.

Díky oddělení prostředí od domény dokáže řídicí harness obsluhovat jak doménu programového kódu (kompilace, testy, typy), tak doménu textové sazby a dokumentace (Typst, LaTeX) shodným stavovým automatem bez nutnosti větvení orchestrační logiky.

4.2 Provozní metriky a spolehlivost agentních běhů

4.3 Poznatky a systémová úskalí z provozu

Praktické nasazení autonomních vývojových agentů do prostředí kontinuální integrace přineslo řadu cenných inženýrských poznatků:

- **Řízení souběžnosti a větvené zámky:** Paralelní integrační běhy vyžadují striktní zámky na úrovni větví, aby nedocházelo ke kolizím a uváznutí workflow.
- **Transparentnost hlášení selhání:** Zákaz tichého pohlcování výjimek; každá chyba nástroje nebo API musí být zaznamenána do kontextu a eskalována člověku. Tiché maskování chyb vede k masivním halucinacím modelu.
- **Deklarativní detekce prostředí:** Vyloučení implicitních předpokladů o repozitáři; veškeré kroky se deterministicky odvozují z přítomnosti souborů a deklarativního manifestu.

4.4 Diskuse: Porovnání architektur

Porovnání navrženého přístupu se současnými referenčními platformami (jako jsou SWE-agent (6), Devin či Copilot Workspace) odhaluje zásadní architektonické rozdíly:

- **Deterministický DAG vs. nekonečná smyčka:**
 - **Volné smyčky (SWE-agent):** Monolitický model v interaktivním terminálu sám rozhoduje o ukončení; při uvíznutí v atraktoru snadno vyčerpá rozpočet tokenů.
 - **Navržený harness:** Model je uzavřen do deterministického grafu (DAG) se striktně alokovaným rozpočtem tahů na každou fázi a izolací kontextu.
- **Granularita lidského dohledu:**
 - **Jednorázový přístup:** Přímé vygenerování celého PR; při špatném pochopení zadání vede ke stovkám řádků vadného kódu a únavě vývojáře z revizí (*review fatigue*).
 - **Dvoufázové schvalování:** Člověk autorizuje záměr a technický plán dříve, než agent začne zasahovat do repozitáře.
- **Bezpečnost a hermetičnost:**
 - **Lokální stroje:** Riziko poškození prostředí či úniku tajemství při spouštění netestovaného kódu.
 - **Efemérní CI kontejnery:** Běh v izolovaném sandboxu s minimálními oprávněními tokenů a auditovatelným protokolem všech operací.

4.5 Systémová a metodická omezení

Přes dosažené výsledky naráží navržená architektura na několik principiálních limitů, které vymezují hranice její současné použitelnosti:

- **1. Hranice deterministické verifikovatelnosti:** Autonomie je spolehlivá pouze u objektivně testovatelných změn (kód, testy, typy). U subjektivních úloh (UX ergonomie, grafický design, stylistická formulace textu) zůstává role modelu asistenční a validaci provádí člověk.
- **2. Propustnost a dostupnost inferenčních API:** Rotační žebříček tlumí lokální výpadky, avšak globální kvóty a latence cloudových poskytovatelů tvoří pevný strop průchodnosti pipeline.
- **3. Absence vícevláknového řešení konfliktů:** Souběžná práce mnoha autonomních agentů a lidí nad týmiž soubory a řešení komplexních merge konfliktů představuje otevřenou výzvu pro navazující výzkum.

5 Závěr

Seznam příloh

A	Obsah přiloženého média	27
B	Schéma konfiguračního manifestu darkfactory.json	28
C	Sdílené workflow pro GitHub Actions	29
D	Systémové prompty plánovacího a kódovacího agenta	30
E	Protokol revizních značek v sazebním systému Typst	31

A Obsah přiloženého média

Odevzdaný archiv obsahuje kompletní zdrojové soubory práce, sazební šablonu, řídicí skripty a zdrojový kód autonomního systému DarkFactory.

```
mono-OdbornaPrace/
├─ prace/                                # Rukopis a sazba odborné práce
│   └─ kapitoly/                         # Texty jednotlivých kapitol (01-uvod
až 06-prilohy)
│   └─ lib/odborna-prace.typ            # Sazební šablona, recenzní značky a
formátování GJKT
│   └─ img/                             # Vektorová procesní schémata (SVG) a
grafické podklady
│   └─ fonts/                           # Metricky shodná patková písma
(Caladea)
│   └─ bib/references.bib               # Bibliografická databáze citovaných
zdrojů
│   └─ scripts/                         # Pomocné a náhledové skripty
│   └─ main.typ                         # Hlavní řídicí dokument sazby
│   └─ metadata.typ                     # Údaje o autorovi, vedoucím a
anotace
│   └─ Makefile                         # Příkazy pro automatizovanou
kompilaci a kontrolu
├─ darkfactory/                         # Zdrojový kód systému DarkFactory
(submodul)
│   └─ .github/workflows/               # Sdílené šablony kontinuální
integrace
│   └─ src/                             # Výkonné orchestrační jádro systému
├─ out/prace.pdf                        # Výsledný vysázený tiskový dokument
v PDF
└─ README.md                           # Dokumentace a návod na reprodukci
prostředí
```

Výpis 1: Stromová adresářová struktura odevzdaného elektronického archivu a doprovodných repozitářů.

B Schéma konfiguračního manifestu darkfactory.json

C Sdílené workflow pro GitHub Actions

D Systémové prompty plánovacího a kódovacího agenta

E Protokol revizních značek v sazebním systému Typst

Tato příloha uvádí referenční definici a použití vizuálních revizních značek pro řízení a dohled nad generovaným textem v sazebním formátu Typst.

```
#import "lib/odborna-prace.typ": note, issue, alert, critique, added,
draft, confirmed, diff

// --- 1. Panely na okraji textu (Callouty) ---
#note[Doplňte porovnání rychlosti kompilace mezi verzemi 0.1 a 0.2.]
#issue[Chybná signatura funkce: chybí povinný parametr timeout.]
#alert[Sekce postrádá shrnutí naměřených výsledků před diskusí.]
#critique[Metodologická absence baseline prokazující přínos nového
modulu.]
#blue-note[Metodické vymezení: rozsah práce a oddělení agentního
inženýrství od strojového učení.]

// --- 2. Textové revizní funkce (zvýraznění v toku textu) ---
#draft[Tento odstavec tvoří neověřený koncept čekající na schválení.]
#added[Nově vygenerovaná sekce automaticky začleněná agentem.]
#confirmed[Uživatelem zkontrolovaný text, který ještě nebyl finalizován.]
#removed[Zastaralý text navržený k odstranění.]
#diff[Původní chybné znění textu.][Nové opravené znění textu po revizi.]
```

Výpis 2: Ukázka zápisu a použití revizních značek a textových funkcí v jazyce Typst.

Vizuální reprezentace jednotlivých prvků v sazbě

Pro přehlednost jsou níže uvedeny reálné ukázky jednotlivých revizních panelů a textových funkcí v jejich finální vizuální podobě:

Ukázky textových zvýrazňovacích a srovnávacích funkcí v toku odstavce:

- **Neověřený koncept** (`#draft` / `#unconfirmed`): Tento text představuje koncept čekající na posouzení autorem.
- **Nově přidáný text** (`#added`): Tato pasáž byla nově vygenerována autonomním agentem na základě požadavku.
- **Potvrzený text** (`#confirmed`): Text byl předběžně odsouhlasen uživatelem, čeká na finální začištění.
- **Navrženo k odstranění** (`#removed`):
- **Srovnávací diff** (`#diff`): Nové přesné, fakticky a formálně ověřené znění pasáže.
- **Čistý neoznačený text**: Představuje finální, autorsky schválený text v hlase autora bez jakéhokoliv podbarvení.

Role značek v lidském dohledu:

- **Čistý neoznačený text**: Schválený autorský text v hlase autora.

- **Žluté podbarvení (#unconfirmed)**: Neověřený koncept čekající na lidské posouzení.
- **Zelené podbarvení (#added)**: Nově vygenerované návrhy agenta.
- **Modré podbarvení (#confirmed)**: Uživatelem odsouhlasený text.
- **Srovnávací diff (#diff)**: Transparentní vizualizace navržených oprav.
- **Postranní panely (Callouty)**: Striktní oddělení námětů, chyb a oponentury od těla textu.

Seznam zdrojů

1. CHACON, Scott a STRAUB, Ben. *Pro Git*. 2. New York : Apress, 2014. ISBN 978-1-4842-0076-6.
2. HUMBLE, Jez a FARLEY, David. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston : Addison-Wesley, 2010. ISBN 978-0-321-60191-9.
3. VASWANI, Ashish, SHAZEER, Noam, PARMAR, Niki, USZKOREIT, Jakob, JONES, Llion, GOMEZ, Aidan N., KAISER, Łukasz a POLOSUKHIN, Illia. Attention Is All You Need. *arXiv preprint arXiv:1706.03762*. Online. 2017. Available from: <https://arxiv.org/abs/1706.03762>
4. LIU, Nelson F., LIN, Kevin, HEWITT, John, PARANJAPE, Ashwin, BEVILACQUA, Michele, PETRONI, Fabio a LIANG, Percy. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*. Online. 2024. Vol. 12, p. 157–173. Available from: <https://arxiv.org/abs/2307.03172>
5. ANTHROPIC. Prompt Engineering overview. Online. 2026. Available from: <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview>
6. YAO, Shunyu, ZHAO, Jeffrey, YU, Dian, DU, Nan, SHAFRAN, Izhak, NARASIMHAN, Karthik a CAO, Yuan. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*. Online. 2022. Available from: <https://arxiv.org/abs/2210.03629>
7. ANTHROPIC. Model Context Protocol documentation. Online. 2026. Available from: <https://modelcontextprotocol.io/docs/2026-07-28/getting-started/intro#explore-mcp>
8. MARIUS, Patrik. DarkFactory: autonomous, governed software engineering pipelines. Online. 2026. [Accessed 8 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory>
9. Deepseek Harness. *arXiv preprint arXiv:2608.25512*. Online. 2026. Available from: <https://arxiv.org/abs/2608.25512>
10. DAO, Tri, FU, Daniel Y., ERMON, Stefano, RUDRA, Atri a RÉ, Christopher. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *Advances in Neural Information Processing Systems*. Online. 2022. Vol. 35, p. 16344–16359. Available from: <https://arxiv.org/abs/2205.14135>
11. AINSLIE, Joshua, LEE-THORP, James, JONG, Michiel de, ZEMLYANSKIY, Yury, LEBRÓN, Federico a SANGHAI, Sumit. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *arXiv preprint arXiv:2305.13245*. Online. 2023. Available from: <https://arxiv.org/abs/2305.13245>